OnDeviceTraining

Code Reading Guide 4

Line-by-Line Syntax & Explanation — Beginner Edition
Files 25–29: Conv1d · MaxPool1d · Flatten · Serialize/Deserialize · Optimizer/SGD

Mohamed · Master's Thesis · 2026

Guide 4 completes the layer internals that were left out of Guide 2. Guide 2 explained how Layer.h dispatches calls polymorphically; Guide 4 shows what actually happens inside each layer's forward and backward functions. It then covers serialization (how trained weights are saved to disk and loaded back), and the full optimizer internals including both plain SGD and SGD with momentum. After reading this guide you will have seen every file needed to understand, train, evaluate, and save a model with the OnDeviceTraining framework.

Contents

Phase 13 — Conv1d Layer Internals

Conv1d.h / Conv1d.c

Conv1d is the computational core of the HAR model — three stacked Conv1d layers extract local patterns from the 128-timestep sensor signal. Conv1d.c implements four responsibilities: **initConv1dConfigWithWeightsAndBias** fills the config struct, **conv1dForward** runs the sliding-window dot-product, **conv1dBackward** computes weight gradients and propagates the loss gradient, and **conv1dCalcOutputShape** computes the output dimensions for tensor allocation.

A 1-D convolution slides a kernel of size K across an input of length L. At each position outPos, the kernel is multiplied element-wise with the input window and the products are summed. With SAME padding the output length equals the input length; with VALID padding it shrinks by K-1.

25 · Conv1d.h / Conv1d.c

src/layer/include/Conv1d.h + src/layer/Conv1d.c

initConv1dConfigWithWeightsAndBias — config setup

`void initConv1dConfigWithWeightsAndBias(`	Fills every field of a conv1dConfig_t struct. The struct is pre-allocated by the caller (typically Conv1dApi.c); this function only writes to it.
`conv1dConfig_t *conv1dConfig, kernel_t *kernel,`	kernel_t describes the convolution geometry: kernel size, stride, padding, dilation. It is passed to windowGeometry1dCalc to compute the output length.
`parameter_t *weights, parameter_t *bias,`	parameter_t pairs a param tensor (the actual weights) with a grad tensor (where backprop accumulates the gradient). weights shape: [outChannels, inChannels/groups, kernelSize]. bias shape: [outChannels].
`quantization_t *forwardQ, *weightGradQ, *biasGradQ, *propLossQ) {`	Four quantization configs: forwardQ sets the output dtype (FLOAT32 in this project), weightGradQ/biasGradQ set the gradient dtype, propLossQ sets the dtype of the gradient propagated to the previous layer.
`if (groups == 0) { PRINT_ERROR(...); exit(1); }`	Guard: groups must be at least 1. groups=1 is standard convolution. groups=inChannels is depthwise convolution. The guard catches a common misconfiguration before a division-by-zero.

conv1dForward — the dispatch function

`void conv1dForward(layer_t *layer, tensor_t *input, tensor_t *output) {`	Public entry point called by the layerFunctions vtable. Does NOT perform the computation directly — it reads the quantization type and dispatches to the appropriate typed implementation.
`switch (cfg->forwardQ->type) {`	Branch on the forward quantization type. Currently only FLOAT32 is implemented. Future variants (INT8, SYM_INT32) would add cases here without changing any calling code.
`case FLOAT32: conv1dForwardFloat(layer, input, output); break;`	For float32 forward: delegate to conv1dForwardFloat, which calls the low-level conv1dKernelFloat32 arithmetic primitive from Conv1dKernel.c.

`void conv1dForwardFloat(layer_t *conv1dLayer, tensor_t *input, tensor_t *output) {`	Thin wrapper: extract the weight tensor and optional bias tensor from the config, then call the kernel. The kernel handles the nested loop over batch, output channel, and output position.
`conv1dKernelFloat32(input, weightTensor, biasTensor, cfg->kernel, cfg->groups, output);`	conv1dKernelFloat32 is in Conv1dKernel.c. It uses windowSlice1dAt to find the input positions that fall under the kernel at each output position, respecting stride, padding, and dilation.

conv1dBackward — weight gradients and propagated loss

The backward pass has two parts: first compute the weight gradient (how the loss changes with respect to each weight), then propagate the loss gradient to the input (how the loss changes with respect to each input position). Both are required for a correct backpropagation chain.

`static void conv1dCalcWeightGradsFloat32(conv1dConfig_t *cfg,`	Private function (static = file-scope only). Computes dLoss/dWeights for every weight element by accumulating over all batch items, output positions, and input channel offsets.
`size_t batch = forwardInput->shape->dimensions[0];`	Read the batch size from the input tensor's shape. Shapes are 3D: [batch, channels, length]. dimensions[0]=batch, [1]=channels, [2]=length.
`windowGeometry1d_t geom = windowGeometry1dCalc(inputLength, cfg->kernel);`	Recompute the window geometry from the input length and kernel config. geom.outputLength must match the gradient's output dimension — the check below validates this.
`float const *xArr = (float const *)forwardInput->data;`	Cast the raw data pointer to float*. tensor_t stores data as uint8_t* (a generic byte buffer) to support multiple dtypes. The cast is safe because forwardQ->type == FLOAT32 guarantees the bytes hold IEEE 754 floats.
`float const *gyArr = (float const *)lossGrad->data;`	Gradient from the next layer (dLoss/dOutput of this layer). Shape: [batch, outChannels, outputLength].
`float *gwArr = (float *)cfg->weights->grad->data;`	Gradient accumulator for the weights. Note: += not = because multiple samples in a batch accumulate into the same gradient buffer. sgdZeroGrad clears this to zero before each batch.
`for (size_t b=0; b<batch; b++) { for (size_t g=0; g<groups; g++) { ...`	Triple nested loop over batch items, groups, and output channels. Groups partition channels into independent sub-convolutions. For groups=1 (standard conv) the group loop runs once.
`windowSlice1d_t slice = windowSlice1dAt(&geom;, outPos);`	Compute the window slice at output position outPos. slice.firstValidInputIdx is the first input index under the kernel. slice.validCount is how many kernel positions overlap valid input (handles boundary padding).
`gwArr[(oc*inChPerGroup+icOffset)*kernelSize+kernelIdx] += xv * gy;`	Core weight gradient update: dLoss/dWeight[oc,ic,k] += input[b,ic,inputIdx] * grad[b,oc,outPos]. This is the standard convolution weight gradient formula. += accumulates over all batch items and output positions.

`convTranspose1dKernelFloat32(lossGrad, cfg->weights->param, NULL, ..., propLoss);`	Propagate gradient to input (dLoss/dInput). A transposed convolution (also called deconvolution or backward conv) with the original weights spreads each output gradient back to the input positions that contributed to it.

conv1dCalcOutputShape — geometry computation

`void conv1dCalcOutputShape(layer_t *conv1dLayer, shape_t *inputShape, shape_t *outputShape) {`	Called during tensor allocation before the forward pass. Fills outputShape so the caller can allocate the output buffer.
`if (inputShape->numberOfDimensions != 3) { ... exit(1); }`	Validate input rank. Conv1d requires exactly 3 dimensions: [batch, channels, length]. 2D or 4D inputs are a user error.
`windowGeometry1d_t geom = windowGeometry1dCalc(inputLength, cfg->kernel);`	Compute output length from kernel config. For SAME padding with stride 1: outputLength = inputLength. For VALID padding: outputLength = (inputLength - kernelSize) / stride + 1.
`outputShape->dimensions[0] = batchSize;`	Batch dimension is unchanged — each sample produces one output window.
`outputShape->dimensions[1] = outChannels;`	Channel dimension becomes the number of filters (outChannels = CONV1_F, CONV2_F, or CONV3_F).
`outputShape->dimensions[2] = geom.outputLength;`	Length dimension depends on kernel size, stride, and padding.

Phase 14 — Pooling and Flatten

MaxPool1d.h / MaxPool1d.c · Flatten.h / Flatten.c

After each Conv+ReLU block in the HAR model, a MaxPool1d halves the sequence length. After the third block, an AvgPool1d reduces each channel to a single value, and then Flatten reshapes the 3-D activation into a 1-D feature vector ready for the linear classifier. These are simpler layers than Conv1d — their forward pass is a selection or averaging operation, and their backward pass is a routing operation.

26 · MaxPool1d.h / MaxPool1d.c

src/layer/include/MaxPool1d.h + src/layer/MaxPool1d.c

MaxPool1d slides a window of size K with stride S across the input and selects the maximum value in each window. During the forward pass it records WHICH input position produced the maximum (the argmax). During the backward pass, the gradient flows only to that saved argmax position — all other positions receive zero gradient.

maxPool1dForwardFloat — forward pass

`void maxPool1dForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output) {`	Implements max pooling for float32 tensors. Input shape: [batch, channels, inputLength]. Output shape: [batch, channels, outputLength] where outputLength = (inputLength - K) / S + 1.
`int32_t *argmaxArr = (int32_t *)cfg->argmaxIndices->data;`	Pre-allocated buffer for the argmax indices. argmaxArr[b,c,outPos] will store the input position index that produced the maximum for that output. This must be kept alive until the backward pass.
`float bestVal = -INFINITY; int32_t bestInputIdx = -1;`	-INFINITY is the IEEE 754 representation of negative infinity (from math.h). Any real float value is greater, so the first input element will always update bestVal. bestInputIdx = -1 is a sentinel for 'no valid index yet'.
`for (size_t i = 0; i < slice.validCount; i++) {`	Iterate over the valid input positions within the window. slice.validCount can be less than K near the boundaries if padding is VALID.
`if (v > bestVal) { bestVal = v; bestInputIdx = (int32_t)inputIdx; }`	Track the winner. Only strict > so ties keep the first (leftmost) winner. bestInputIdx is cast to int32_t so it fits in the argmaxIndices tensor.
`yArr[outIdx] = bestVal; argmaxArr[outIdx] = bestInputIdx;`	Write results. yArr gets the maximum value; argmaxArr gets the position index that produced it — saved for the backward pass.
`yArr[outIdx] = 0.0f; argmaxArr[outIdx] = -1; PRINT_ERROR(...);`	Fallback for an empty window (slice.validCount == 0). In practice this cannot happen with well-configured padding, but the code handles it gracefully rather than crashing or producing undefined behaviour.

maxPool1dBackwardFloat — gradient routing

Max pooling has no learnable parameters, so there is no weight gradient. The backward pass only routes the incoming gradient to the position that won the max competition during the forward pass.

`void maxPool1dBackwardFloat(layer_t *layer, tensor_t *forwardInput,`	forwardInput is explicitly unused (marked void to suppress compiler warning). The argmax tensor already encodes which input position to update — re-reading the input would be wasteful.

`(void)forwardInput;`	Explicit void cast — tells the compiler 'I know this parameter is unused, this is intentional'. Without this, the compiler would warn about an unused parameter.
`int32_t const *argmaxArr = (int32_t const *)cfg->argmaxIndices->data;`	Read the argmax indices saved during the forward pass. const here prevents accidental writes to the saved indices.
`gxArr[(b*channels+c)*inputLength+(size_t)inputIdx] += gyArr[outIdx];`	Route the output gradient to the winning input position. All other positions in this window get zero gradient because they did not contribute to the output (they were not the maximum). += accumulates in case of overlapping windows (stride < kernel size).
`if (inputIdx < 0) { continue; }`	Skip empty-window sentinels. inputIdx = -1 means no valid input contributed to this output position, so no gradient should flow.

27 · Flatten.h / Flatten.c

src/layer/include/Flatten.h + src/layer/Flatten.c

Flatten is the simplest layer in the framework. Its forward and backward passes are both just a memcpy — the data bytes are identical, only the shape metadata changes. After the AvgPool1d in the HAR model, the activation is [batch=64, channels=64, length=1]. Flatten reshapes this to [batch=64, features=64] so the Linear layer sees a 1-D feature vector per sample.

`void flattenForward(layer_t *flattenLayer, tensor_t *input, tensor_t *output) {`	The flattenLayer argument is unused — Flatten has no configuration. The (void)flattenLayer cast below suppresses the compiler warning.
`(void)flattenLayer;`	Suppress unused-parameter warning. Same idiom used in maxPool1dBackwardFloat.
`size_t numberOfElements = calcNumberOfElementsByTensor(input);`	Count the total number of scalar values: batch * channels * length = 64 * 64 * 1 = 4096 elements for the HAR model.
`memcpy(output->data, input->data, numberOfBytes);`	Copy the raw bytes. memcpy is safe here because the number of elements is identical before and after flattening — only the shape interpretation changes. The output tensor has a different shape metadata but the same data bytes.
`if (input->quantization->type == SYM_INT32) { outputQC->scale = inputQC->scale; }`	Propagate the quantization scale to the output. For float32 there is no scale to copy, but for INT32 the scale factor must be forwarded so the following Linear layer knows how to interpret the values.
`void flattenBackward(...) { (void)flattenLayer; (void)forwardInput;`	Both flattenLayer and forwardInput are unused in backward too. The backward pass is also a memcpy: gradients pass through Flatten unchanged.
`void flattenCalcOutputShape(layer_t *flattenLayer, shape_t *inputShape, ...) {`	Compute the 2-D output shape. Everything except the batch dimension is multiplied together into a single features dimension.
`size_t features = 1; for (size_t i=1; i<inputShape->numberOfDimensions; i++) { features *= ...; }`	Compute the product of all non-batch dimensions. For [64, 64, 1]: features = 64 * 1 = 64. For [64, 32, 4]: features = 32 * 4 = 128.

`outputShape->dimensions[0] = batch;`
`outputShape->dimensions[1] =`
`features;`
	Output is always 2-D: [batch, features]. numberOfDimensions is set to 2 regardless of the input rank.

Phase 15 — Serialization

Serialize.h / Serialize.c · Deserialize.h / Deserialize.c

Serialization saves the trained model (weights, biases, quantization parameters) to a binary file so it can be loaded back without retraining. The binary format is extremely compact — no headers, no metadata, just the raw bytes of each tensor written in a fixed traversal order that matches the deserialization code exactly. Serialize.c and Deserialize.c are mirror images of each other: every fwrite in Serialize has a matching fread in Deserialize.

28 · Serialize.h / Serialize.c

src/serial/include/Serialize.h + src/serial/Serialize.c

The serialization entry points are three public functions: `serializeTensor`, `serializeParameter`, and `serializeModel`. All helper functions are static (file-private). Everything ultimately calls the single primitive `serialize(values, count, size, f)` which wraps fwrite.

`static void serialize(void *values, size_t n, size_t sz, FILE *f) {`	The only function that touches the file. void* accepts any pointer type. n is the number of items, sz is the size of each item in bytes. fwrite writes n*sz bytes to f. All other functions call this.
`fwrite(values, numberOfElements, sizeOfElement, f);`	fwrite returns the number of items successfully written. This implementation does not check the return value — a production version would assert that the return equals numberOfElements to catch disk-full errors.
`void serializeTensor(tensor_t *tensor, FILE *f) {`	Serialize one tensor in fixed order: shape, quantization config, raw data bytes. The deserializer reads them back in the same order to reconstruct the tensor.
`serializeShape(tensor->shape, f);`	Writes numberOfDimensions (size_t, 8 bytes on 64-bit), then dimensions[0..N-1] (N * 8 bytes), then orderOfDimensions[0..N-1] (N * 8 bytes).
`serializeQuantization(tensor->quantization, f);`	Writes the qtype_t enum (4 bytes), then the quantization config if the type has one. FLOAT32 has no qConfig so only the enum is written. SYM_INT32 also writes scale (float, 4 bytes), rounding mode, and qMaxBits.
`serializeData(tensor->data, numberOfValues, bytesPerValue, f);`	Writes the raw data buffer. bytesPerValue is 4 for FLOAT32, 4 for INT32/SYM_INT32. totalBytes = numberOfValues * bytesPerValue.
`serializeSparsity();`	Currently a no-op (empty function body). Sparsity metadata (e.g. the RigL mask) is not yet serialized. Marked TODO in the source.
`void serializeParameter(parameter_t *parameter, FILE *f) {`	Serialize a parameter: first the param tensor (the weights), then the grad tensor (the gradient accumulator). Both are tensors with the same shape; only their data values differ.
`void serializeModel(layer_t **model, size_t sizeModel, FILE *f) {`	Iterate over every layer and call serializeLayer for each one. The order must match the model array exactly, because deserializeModel reads in the same order.
`static void serializeLayer(layer_t *layer, FILE *f) {`	Switch on layer type. Each case serializes the layer-specific config. FLATTEN has no state (no weights, no quantization) so its case is empty. Unsupported types call exit(1) — a future Conv1d case would be added here.

`case LINEAR: serializeParameter(linearConfig->weights, f); ...`	For a Linear layer: weights parameter, bias parameter, then the four quantization configs (forwardQ, weightGradQ, biasGradQ, propLossQ). This exact sequence is reversed in deserializeLayer.
`case RELU: serializeQuantization(reluConfig->forwardQ, f); ...`	For ReLU and Softmax: only the quantization configs are saved — these layers have no learnable parameters.
`case FLATTEN: break;`	Flatten has no state whatsoever, so this case is an empty break. A zero-byte contribution to the file from this layer type.

28 (continued) · Deserialize.h / Deserialize.c

src/serial/include/Deserialize.h + src/serial/Deserialize.c

Deserialize.c is structurally identical to Serialize.c with fwrite replaced by fread. The traversal order is byte-for-byte identical so both files always stay in sync. One important difference: deserializeTensor reads the shape dimensions count first (to know how many bytes follow), whereas serializeTensor writes it. The tensor must already exist (its struct allocated) before deserialization — only the data inside it is overwritten.

`static void deserialize(void *values, size_t n, size_t sz, FILE *f) {`	Mirror of serialize. fread reads n items of sz bytes each from f into values. Like fwrite, the return value is not checked in this implementation.
`void deserializeTensor(tensor_t *tensor, FILE *f) {`	Read shape, quantization, data — in the same order they were written. The tensor must already be allocated (shape arrays must have space), because deserializeShape writes into tensor->shape->dimensions.
`static void deserializeShape(shape_t *shape, FILE *f) {`	Reads numberOfDimensions first (so the subsequent array reads know their length), then dimensions[0..N-1], then orderOfDimensions[0..N-1].
`static void deserializeQConfig(quantization_t *q, FILE *f) {`	Switch on q->type (just read by deserializeQuantization) to determine what additional bytes to read. SYM_INT32 reads scale, roundingMode, qMaxBits in the same order Serialize.c wrote them.
`static void deserializeSparsity() {}`	No-op, matching serializeSparsity. The mask array is NOT loaded from the file. After loading a model, the RigL masks must be re-initialised separately.

Phase 16 — Optimizer Internals

Optimizer.c · Sgd.c (plain SGD and SGD with momentum)

The optimizer applies the weight update rule after each batch. Optimizer.c provides the dispatch table and a helper to count optimizer states. Sgd.c implements four functions: **sgdStep** (vanilla SGD), **sgdStepM** (SGD with momentum), **sgdZeroGrad** (reset all gradients), and **sgdInit** (fill the sgd_t config). Each step function has a float32 and a SYM_INT32 variant — the INT32 variant converts to float, updates, then converts back.

29 · Optimizer.c — the dispatch table

src/optimizer/Optimizer.c

`optimizerFunctions_t optimizerFunctions[] = {`	Global array of function-pointer structs. Indexed by optimizer type (SGD=0, SGD_M=1). This is the same dispatch table pattern used by layerFunctions in Layer.c.
`[SGD] = {sgdStep, sgdZeroGrad},`	C99 designated initialiser. [SGD] sets element at index SGD (the enum value for plain SGD). Stores two function pointers: step=sgdStep and zero=sgdZeroGrad.
`[SGD_M] = {sgdStepM, sgdZeroGrad}};`	SGD with momentum shares the same zero function but uses sgdStepM which reads and updates the velocity state buffers.
`static size_t calcNumberOfStatesByLayerType(const layerType_t type) {`	Returns how many optimizer state tensors a layer needs. LINEAR and CONV1D need 2 states (weights + bias parameter). RELU, SOFTMAX, FLATTEN, MAXPOOL1D need 0 (no trainable parameters).
`size_t calcTotalNumberOfStates(layer_t **model, size_t sizeModel) {`	Sum up the state count across all layers. Used to allocate the optimizer->states array. Called once during optimizer initialisation.

29 (continued) · Sgd.c — sgdInit and sgdStep (plain SGD)

src/optimizer/Sgd.c

Vanilla SGD (stochastic gradient descent) applies the simplest possible weight update: weight -= learningRate * gradient. It has no memory of previous updates.

`void sgdInit(sgd_t *sgd, float learningRate, float momentumFactor, float weightDecay) {`	Fills the sgd_t config struct. learningRate (LR) scales the gradient before applying it to the weight. weightDecay adds a penalty proportional to the weight magnitude (L2 regularisation). momentumFactor is only used by sgdStepM.
`static void sgdStepFloat(optimizer_t *optim) {`	Private, float32 implementation of plain SGD. Called by sgdStep when optimizer->qtype == FLOAT32.
`for (size_t stateIndex = 0; stateIndex < optim->sizeStates; stateIndex++) {`	Iterate over every parameter managed by this optimizer. Each Conv1d layer contributes 2 states (weights, bias). Each Linear layer contributes 2 states. Total for the HAR model: 8 states.
`float *gradArr = (float *)param->grad->data;`	Gradient buffer: dLoss/dWeight, accumulated by CalculateGradsSequential over the current batch. This is what the optimizer consumes.
`float *dataArr = (float *)param->param->data;`	The actual weight values, updated in place.

`float grad = gradArr[elementIndex] + sgd->weightDecay * dataArr[elementIndex];`	L2 regularisation: add weightDecay * w to the gradient. This is equivalent to adding a term 0.5 * weightDecay * ||w||^2 to the loss. For weightDecay=0, this line simplifies to grad = gradArr[elementIndex].
`dataArr[elementIndex] -= sgd->learningRate * grad;`	The actual weight update: w = w - lr * grad. The minus sign is because we want to move in the direction of steepest descent (gradient points up, we want to go down).

29 (continued) · sgdStepM — SGD with momentum

SGD with momentum introduces a velocity term: instead of applying the gradient directly, the optimizer maintains a running average of past gradients (the momentum). This dampens oscillations and accelerates convergence. The update rule is:

```
velocity = momentumFactor * velocity + gradient (with weight decay applied to gradient)
weight -= learningRate * velocity
```

The HAR+RigL project uses SGD_M with momentum=0.9, meaning 90% of the previous velocity is retained each step.

`static void sgdStepMFloat(optimizer_t *optim) {`	Float32 implementation of momentum SGD. Needs the velocity state buffer in addition to gradients and weights.
`states_t *states = optim->states[i];`	states_t is a struct holding the velocity tensor(s) for parameter i. For Conv1d and Linear layers, states[0] holds the weight velocity; states[1] would hold the bias velocity if bias is tracked separately.
`tensor_t *state = states->stateBuffers[0];`	The velocity tensor — same shape as the weight tensor. Initialised to zeros before training begins. Persists across batches (unlike the gradient, which is zeroed after each batch).
`float *stateArr = (float *)state->data;`	Pointer to the velocity values. Updated in place each step.
`stateArr[elementIndex] = sgd->momentumFactor * stateArr[elementIndex] + grad;`	Velocity update: v = m*v + g. The momentum factor (0.9) blends the current gradient with the accumulated past velocity. With m=0.9, the effective learning rate is amplified in directions of consistent gradient.
`paramArr[elementIndex] -= sgd->learningRate * stateArr[elementIndex];`	Weight update: w = w - lr * v. Uses the updated velocity, not the raw gradient.
`static void sgdStepMSymInt32(optimizer_t *optim) {`	INT32 variant: convert param, grad, AND state to float, apply the update in float arithmetic, then convert all three back to INT32. Three conversions per parameter instead of one — more expensive but necessary to preserve numerical accuracy across quantised types.

29 (continued) · sgdZeroGrad — reset gradients

`void sgdZeroGrad(optimizer_t *optimizer) {`	Called after every optimizer step to clear the gradient buffers. Must be called before the NEXT batch starts accumulating gradients.
`size_t bitsPerElement = calcBitsPerElement(param->grad->quantization);`	Compute the gradient buffer size. FLOAT32 = 32 bits per element, INT32/SYM_INT32 = 32 bits. The calculation: totalBytes = ceil(numValues * bitsPerElement / 8).

`memset(param->grad->data, 0, totalNumberOfBytes);`	Zero all bytes in the gradient buffer. memset is a standard C function that fills a memory region with a single byte value (0 here). Setting all bytes to 0 produces 0.0f for float32 and 0 for int32.
`if (param->grad->quantization->type == SYM_INT32) { symIntQ->scale = 1.f; }`	Reset the scale factor to 1.0 for INT32 gradients. After a backward pass, the scale may have been set to a non-unity value during quantisation-aware arithmetic. Resetting it ensures the next batch starts with a fresh scale.

Syntax Quick Reference — Guide 4 Additions

Syntax / Pattern	Full name / Meaning	Where used in Guide 4
(void)unusedParam;	Explicit unused-parameter cast. Suppresses compiler 'unused parameter' warning.	Flatten, MaxPool1d backward
float bestVal = -INFINITY;	-INFINITY = IEEE 754 negative infinity (requires math.h). Any finite value compares greater.	MaxPool1d forward — max initialisation
fwrite(buf, n, sz, f)	Write n items of sz bytes from buf to file f. Returns items written (not checked here).	Serialize.c — all write operations
fread(buf, n, sz, f)	Read n items of sz bytes from f into buf. Mirror of fwrite.	Deserialize.c — all read operations
[SGD]={fn1,fn2}	C99 designated initialiser for an array. [SGD] sets the element at index equal to the enum value SGD.	Optimizer.c dispatch table
weight -= lr * grad	SGD update rule. -= means weight = weight - lr*grad. In-place subtraction.	Sgd.c sgdStepFloat
v = m*v + g; w -= lr*v	Momentum update rule. v is velocity, m is momentum factor, g is gradient, lr is learning rate.	Sgd.c sgdStepMFloat
memset(ptr, 0, n)	Fill n bytes starting at ptr with zero. Zeroes a float array when all bytes are 0 (IEEE 754: 0.0f = 0x00000000).	sgdZeroGrad
(float const *)tensor->data	Cast uint8_t* to float const*. const means the floats will not be written through this pointer.	Conv1d, MaxPool1d forward read-only arrays
(float *)tensor->grad->data	Cast to non-const float*. Used for += gradient accumulation.	Conv1d weight gradient loop
gwArr[idx] += xv * gy	Accumulate weight gradient. += because multiple batch samples and output positions contribute to the same weight's gradient.	conv1dCalcWeightGradsFloat32
static void fn(...)	File-private function. Not visible outside this .c file. Used for all helper functions in Serialize, Deserialize, Conv1d.	All files in Guide 4

After Guide 4 you have read every file needed to build, train, evaluate, and save a model. Remaining files (AvgPool1d, Conv1dTransposed, LayerNorm, Dropout, user API wrappers) follow the same patterns established here.